Zewail City of Science and Technology — مدينة زويل للعلوم والتكنولوجيا

Course Project: SW 302, User Interface Development

Project Objective

The objective of the course project is to enable students to design and implement a complete frontend web application that demonstrates their ability to apply **modern UI development practices**. Students will work in groups to:

- Define and scope a project idea (from the suggested list) that addresses a practical problem or simulates a real-world application.
- Design the user interface and user experience (UI/UX) through wireframes and mockups before implementation.
- Develop the frontend using HTML5, TailwindCSS, and React to create a responsive, accessible, and engaging user interface.
- Implement dynamic data handling by integrating with either a public API (if suitable for the chosen project) or a mock API (JSON Server, MockAPI, or local JSON data).
- Build interactive dashboards with charts and visualizations to present data insights.
- Collaborate as a team, practicing version control with GitHub, effective project planning, and clear task distribution.
- Deliver the project in phases, culminating in a polished, functional prototype and a final working frontend system, along with a project presentation.

Project Phases Timeline

Project Phases	Deliverables	Grade (%) of 20	Deadline
Group Team & Project Idea	Students form groups, select a project idea, and review the high-level requirements.	**One grade penalty if not submitted on time**	Monday 12/Oct/2025: 11:59 PM
UI/UX Design & Static Prototype	Design the system's user interface and user experience. Deliver wireframes/mockups and a static prototype (HTML + Tailwind, no backend).	**25 % (5 Grades)**	Monday 26/Oct/2025: 11:59 PM
Core React Components	Build the main React components and navigation flow. Focus on reusable, responsive design.	**30% (6 Grades)**	TBD
Advanced Frontend Features & Data Integration	Add dynamic behaviors and work with real or simulated data.	**30 % (6 Grades)**	TBD
Final Delivery & Presentation	Deliver the complete full-stack system, final report, and group presentation.	**15% (3 Grades)**	TBD

Project Phases Details:

Phase 1: Group Team & Project Idea

- **Objective:** Establish the project's foundation.
- **Tasks:**
  - Form groups of **3–4** students.
  - Select a project idea (from the suggested list).
- **Deliverables:**
  - Select Project Idea
  - List of Team Members.
  - Create a GitHub repository and share access to it with your assigned TA.

Phase 2: UI/UX Design & Static Prototype

- **Objective:** Design and validate the user interface and experience.
- **Tasks:**
  - Create wireframes and mockups for at least **8 key pages** (see list below).
  - Build a static prototype using HTML5 + Tailwind CSS (no interactivity).
  - **You must install Tailwind to your project and don't use the CDN version.**
  - Ensure accessibility and responsiveness (desktop & mobile) for your pages.
  - Every page must have both a desktop version and a mobile version.
  - **Do the Required Pages (minimum set):**
    - **Landing / Home Page** – introduction or entry point.
    - **Dashboard Page** – with at least one chart or visualization.
    - **List/Overview Page** – browse or view all items (transactions, recipes, or movies).
    - **Detail Page** – display details of a single item.
    - **Form Page** – to add/edit data (e.g., add expense, recipe, or movie).
    - **Authentication Pages** – Login and Register.
    - **Two Extra Pages** – Such as: Profile, settings, or project-specific feature.
- **Deliverables:**
  - Wireframes/mockups (desktop + mobile).
  - Your delivered pages must show good support for WCAG standards, ARIA attributes, good color contrast, and spatial consistency.
  - Static frontend prototype (HTML + Tailwind).
  - All details described above in this phase's tasks.
  - Make sure to have a complete GitHub repo as proof of your work and commit history as described in this document.

Phase 3: Core React Components

- **Objective:** Develop the functional frontend with React.
- **Tasks:**

[Dr.Rakha will add details and requirements of this Phase later as more course materials are covered.]

Phase 4: Advanced Frontend Features & Data Integration

- **Objective:** Add dynamic behaviors and work with real or simulated data.
- **Tasks:**

[Dr.Rakha will add details and requirements of this Phase later as more course materials are covered.]

Phase 5: Final Delivery & Presentation

- **Objective:** Complete and present the final project.

[Dr.Rakha will add details and requirements of this Phase later as more course materials are covered.]

Team Formation Phase:

- The number of students per team is **3 to 4 students**.
- Each group will assign a representative student as the primary point of contact for communication with TAs.
- The deliverables discussion will take place in the lab after the online submission deadlines.

Note: Workload distribution should be vertical. A student is expected to work across all layers in the software engineering life cycle.

Project Ideas & Detailed Requirements

1: Fitness & Workout Planner

Overview: A web application for users to track workout routines, log completed exercises, and analyze physical activity metrics over time.

Frontend Requirements (React + Tailwind + HTML5)

- Dashboard:
  - Summary cards: Total workouts completed, active streak (days), total estimated calories burned.
  - Bar chart: Weekly workout duration or calories burned per day.
  - Pie/Doughnut chart: Exercise distribution by muscle group (e.g., Chest, Back, Legs, Cardio).

- Exercise Library & Search:
  - Search bar with debounced keyword lookup.
  - Filters: Muscle group, equipment required, difficulty level.
  - Paginated grid of exercise cards with exercise image/gif and quick-add actions.
- Exercise Details:
  - Detailed instruction steps, target muscles, and safety tips.
  - Embedded instructional video or media preview.
  - "Add to Custom Routine" modal with set/repetition inputs.
- Routine Builder & Log:
  - Build and order custom workout routines.
  - Log active workouts with interactive set completion checkboxes and rest timer.
  - Export completed workout history as CSV or JSON.
- Responsive UI:
  - Touch-friendly controls for mobile use in a gym setting.
  - Collapsible navigation drawer and dark mode toggle.

Data Handling Options

- Integrate a public API such as Wger Exercise API or ExerciseDB.
- Alternative: Use a mock API (JSON Server) pre-loaded with exercise schema.
- Persist user routines and workout history in localStorage.

---

2. Event Discovery & Itinerary Planner

Overview: A platform where users discover local events, filter activities, view location details, and curate custom travel or weekend itineraries.

Frontend Requirements (React + Tailwind + HTML5)

- Dashboard:
  - Summary stats: Upcoming events booked, total events saved, spending summary for ticketed events.
  - Line chart: Monthly events attended over the year.
  - Pie chart: Saved events categorized by type (e.g., Music, Tech, Sports, Arts).
- Event Search & Filtering:
  - Search bar supporting event name or venue lookup.
  - Filters: Date range, category, price range (free vs. paid), and location/city.
  - Paginated event listing with status badges (e.g., "Selling Fast", "Free").
- Event Details:
  - Event poster, description, schedule, venue address, and ticket pricing.
  - Interactive map element or embedded map view for venue locations.
  - "Save to Itinerary" and "Mark as Attending" action buttons.
- Itinerary Manager:
  - Drag-and-drop or ordered timeline view of events scheduled by day/time.

  - Conflict warning indicator if two events overlap in time.
  - Export itinerary as a printable PDF or downloadable .ics calendar file.
- Responsive UI:
  - Adaptive layout switching between list view and map/grid layout.
  - Accessible keyboard navigation for interactive timeline components.

Data Handling Options

- Integrate a public API such as Ticketmaster API or PredictHQ API.
- Alternative: Use a mock API simulating event collections.
- Persist saved events, user ratings, and customized itineraries in localStorage.

---

3. Developer Job & Application Tracker

Overview: A career tracking platform allowing software engineering students or job seekers to search tech roles, manage their job application pipeline, and track interview stages.

Frontend Requirements (React + Tailwind + HTML5)

- Dashboard:
  - Key metrics: Total applications submitted, interviews scheduled, offer rate percentage.
  - Funnel / Bar chart: Applications by stage (Applied → Screening → Interview → Offer → Rejected).
  - Pie chart: Distribution of jobs by employment type (Full-time, Remote, Internship, Contract).
- Job Board Search:
  - Search by job title, key skills, or tech stack keywords.
  - Filters: Remote status, experience level, salary range, location.
  - Paginated job board listing with quick save actions.
- Job Details:
  - Full job description, required skills list, company background, and estimated salary range.
  - "One-Click Add to Tracker" form pre-filling company name and job title.
- Application Pipeline Kanban / Board:
  - Kanban columns or categorized list (Saved, Applied, Interviewing, Offered, Closed).
  - Move applications between stages (drag-and-drop or dropdown status updates).
  - Form validation for adding interview notes, contact emails, and follow-up dates.
  - Export application log to CSV for personal record-keeping.
- Responsive UI:
  - Clean dashboard aesthetic with status badge color-coding.
  - Mobile view converting Kanban columns into tabbed views for smaller screens.

Data Handling Options

- Integrate a public job search API such as Adzuna API or JSearch API.
- Alternative: Use a mock API (JSON Server) with structured tech job listings.
- Store personal application pipelines, notes, and saved postings in localStorage.
    -

Charting Technologies for React Dashboards

1. **Recharts**
    - React-specific charting library.
    - Can create bar charts, line charts, pie charts, area charts, etc.
    - https://recharts.org
2. **Chart.js (with react-chartjs-2)**
    - Widely used, well-documented.
    - Offers more customization than Recharts.
    - Great for responsive charts. https://www.chartjs.org

Project Idea Selection:

Students are required to select **one** of the suggested project ideas provided .

Each project must satisfy the following criteria:

1. **Feasibility**
    - `Must be achievable within the course timeline (≈ 10–12 weeks of active work).`
    - The scope should align with the group size and available skills.
2. **Creativity & Practicality**
    - While the ideas are predefined, groups should bring their own creativity into UI/UX design, features, and presentation.
    - The project should simulate a real-world application that users would find useful.
3. **Technical Requirements**
    - Must include both frontend (React + Tailwind + HTML5) and backend (public APIs or mock APIs)
    - A dashboard with at least two chart visualizations is mandatory.
    - When external API integrations are not possible, all data and models should be designed by the students.

GitHub Repository Structure

Each team must maintain a **single GitHub repository** for their project.

The repository must follow a clear structure, with at least the following folders:

- **/docs** → Contains all submitted documents (Proposal, UI/UX Design, Reports, Testing Evidence, etc.).
- **/frontend** → Contains all frontend source code (React + Tailwind + HTML5).
- **/backend** → Contains all mock data and code for API integration
- **/tests** → Contains test cases (unit tests, integration tests, and/or manual test reports if applicable).
- **/deployment** → Contains deployment instructions, configuration files, and environment setup scripts (e.g., Docker, environment variables, DB setup).

Additionally, a **README.md** file must be present in the root of the repository, clearly describing:

- Project title and team members.
- Short project description.
- Instructions for setting up and running both frontend and mock data (backend).
- Links to relevant documents inside /docs.

Commit Guidelines:

- Commits must be meaningful (e.g., "Added dashboard chart component" instead of "fix").
- Commit history should reflect contributions from **all team members**.

Important Rules:

1. The number of students per team is **3 to 4 students**.
2. Teams are encouraged to be from the same lab. Team members can be from different LABS, but NOTE that: It will be your FULL RESPONSIBILITY to attend the evaluation session. Any student who misses the scheduled evaluation session will automatically receive ZERO.
3. Projects are not allowed to be a reuse of another course project.
4. If a student is retaking this course, they must select a new project idea different from their previous one.
5. For each phase, **if a late submission is permitted, a 25% penalty will be incurred** for each day after the deadline. After two overdue days, no submissions will be accepted.
6. Each phase's deliverables must be uploaded to the **Moodle**/or **Google Classroom** assigned assignment and pushed to the repository **before the submission deadline**.
7. The deliverable version uploaded to the **Moodle/or Google Classroom** assignment is the one that will be evaluated. The GitHub version will just be a reference to track progress.
8. **Version Control:** All projects must use GitHub to track progress. Commits should reflect contributions from all members. **GitHub is mandatory in the evaluation process starting Phase-2** and must follow the "GitHub Repository Structure" mentioned. Failing to maintain an updated GitHub repository for your projects may result in a penalty of **up to 20%** for the phase under evaluation.
9. All team members are expected to contribute fairly across the project. Unequal contribution may result in individual grade deductions.
10. **Although this is a team project, every member is expected to fully understand all submitted work. TAs may ask any student about any part of the deliverable** (designs, code, documentation, etc). Any

student unable to explain their project/code may lose marks individually, even if the deliverable was submitted.

11. **Cheating** could result in a negative grade (up to - 10 Course Grades).
12. Each phase submission (when needed) must include **clear documentation** as required.
13. Auto-code generation tools (such as UI Design to Code or System Design to Code, or Diagrams to Code) are not permitted for use and will be considered **cheating**.
14. A **final presentation and live demo** are mandatory (Failing to do so may lead to a big % penalty).
15. **Academic Integrity Reminder**: Directly copying or pasting content from AI tools (such as ChatGPT or others) into your course deliverables is considered academic misconduct (Up to cheating). Your submitted work should reflect your own effort, writing style, creativity, and uniqueness. You may use AI tools to support your learning or improve your understanding, but the final content must be genuinely your own. All submissions/deliverables are subject to plagiarism and AI-detection checks. If there is reasonable suspicion of AI-generated or copied content, the lecturer/TA reserves the right to:
    - Assign a penalty for **cheating** if misuse is confirmed.
    - Request additional work or conduct an oral examination to verify authorship and understanding of the submitted material.
16. The project must be implemented using the mentioned technologies in the project description.

Bonus Opportunities (Optional)

1. [Dr.Rakha may or may not add details and requirements of this Phase later as more course materials are covered.]

Assessment Rubric: Phase 2 – UI/UX Design & Static Prototype

Total Score: 100 Points scaled later to the Phase grade

Category	Weight / Points	Core Objective
1. Technical Understanding of Details	**30 Points**	Understanding the design rationale, layout, and the technical implementation details.
2. Technical Setup & Tailwind CSS Architecture	**10 Points**	Proper Tailwind build setup (NO CDN), semantic HTML5, clean code organization.
3. Page Completeness & Scope	**35 Points**	Delivery of all 8 required pages (Desktop + Mobile versions).
4. Responsiveness & Adaptive Design	**15 Points**	Seamless breakpoint adaptivity, mobile-first designs, layout transformations for all pages.
5. Accessibility (a11y) & Design System Polish	**10 Points**	WCAG standards, ARIA attributes, color contrast, spatial consistency.

Evaluation Criteria by Phase

[Dr.Rakha will update this section later as more course materials are covered.]

Phase	Weight	Evaluation Criteria

Phase 1: Group Team & Project Idea	**- 1 Grade is not done**	- Project idea relevance and feasibility (**2%**) - Clear team formation and GitHub repo created (**2%**) - Professional communication and timely submission (**1%**)
Phase 2: UI/UX Design & Static Prototype	**25%**	- Completeness of required pages (7 pages, desktop + mobile) (**5%**) - Quality of wireframes/mockups (clarity, consistency, alignment with requirements) (**2.5%**) - Static prototype in HTML5 + Tailwind, accuracy to design (**15%**) - Accessibility and responsive design (desktop & mobile) (**2.5%**)
Phase 3: Core React Components	**30%**	- Implementation of reusable React components (forms, buttons, layouts, etc.) (**15%**) - Proper use of React Router for navigation (**5%**) - Basic state management (form handling, UI interactions) (**5%**) - Code quality, responsiveness, and adherence to UI design (**5%**)
Phase 4: Advanced Frontend Features & Data Integration	**30%**	- Correct integration with public APIs or mock API (JSON Server, etc.) **(10%)** - Proper data fetching and rendering in UI **(10%)** - CRUD functionality on frontend (create, update, delete in state or via mock API) **(5%)** - Error handling and loading states **(5%)**
Phase 5: Final Delivery & Presentation	**15%**	- Fully working full-stack application (**4%**) - Project report (clarity, completeness, 5–7 pages) (**3%**) - Presentation/demo (clarity, teamwork, time management, audience engagement) (**3%**)